Unit – 1:

PL/SQL Basics

Prepared By: Prof. Meghna K. Bhatt

fppt.com

Course content

- Brief introduction to SQL Fundamentals
- Basics of PL/SQL
- Advantages of PL/SQL
- PL/SQL Block Structure
- PL/SQL Fundamentals:
  - PL/SQL Variables and PL/SQL Data Types
  - Variable Attribute (%type, %rowtype)
- PL/SQL Control Structure:
  - Conditional Control
  - Iterative Control and Sequential Control.

fppt.com

What is SQL?

- SQL stands for structured query language.
- It was developed by IBM in 1970.
- It is a standard language used to interact with modern RDBMS.
- SQL is a cross-platform and cross-product language.
- It is known as database language which is used to interact with database.
- SQL is English language, it has simple command set.
- It’s a statement & syntax oriented language
- SQL is non-procedural language.
- SQL is used to Creating / maintaining , Manipulating data & Controlling database.

fppt.com

What is SQL?

- SQL is so much powerful in handling data and various database objects.
- But, there is lack some of basic functionalities provided by other programming language.
- For example, SQL does not provide basic procedural capabilities such as conditional checking, branching and looping.
- Oracle provides PL/SQL (Procedural Language / Structured Query Language) to overcome disadvantages of SQL.
- PL/SQL supports all the functionalities provided by SQL along with its own procedural capabilities.
- Data definition statements are not allowed because PL/SQL code is compile time. So, it cannot refer to objects that do not yet exist.

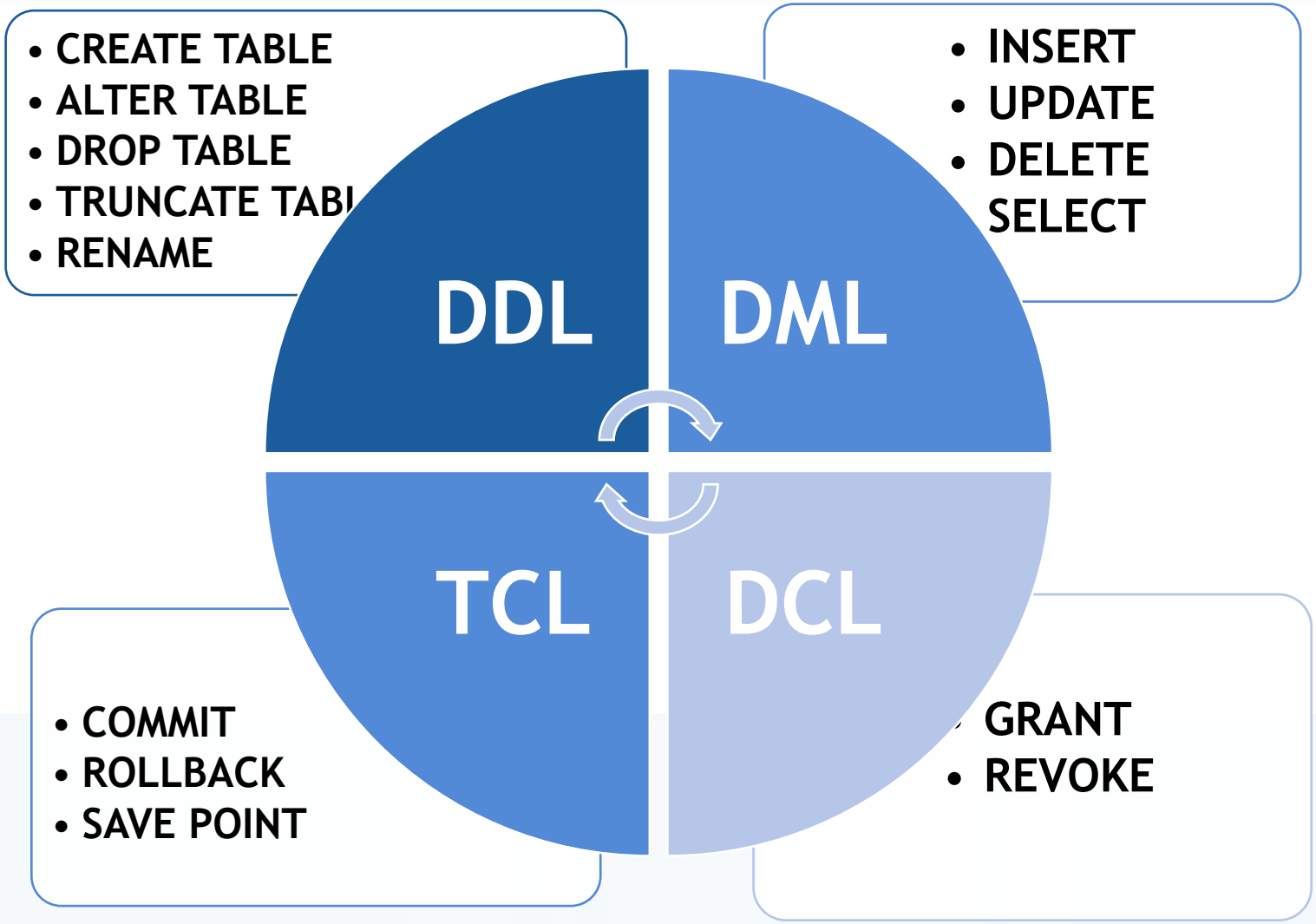
fppt.com

Advantages Of SQL

- **SQL is widely popular because it offers the following –**
  - Allows users to access data in the relational database management systems.
  - Allows users to describe the data.
  - Allows users to define the data in a database and manipulate that data.
  - Allows to embed within other languages using SQL modules, libraries & pre-compilers.
  - Allows users to create and drop databases and tables.
  - Allows users to create view, stored procedure, functions in a database.
  - Allows users to set permissions on tables, procedures and views.

fppt.com

SQL COMMANDS



fppt.com

CREATE TABLE

- Used to create a new table in current user.

syntax:

```
CREATE TABLE  <table name>
(
   <field name>  <Data type>(<Size>)  [CONSTRAINT ],
  <field name>   <Data Type> (<Size>)  [CONSTRAINT ],
   ……
 );
```

Example :

```
CREATE TABLE  student
 (
                    rollno number (2),
                    name  char (8),
                    birthdate date,
                    stream  char(5)
  );
```

ippt.com

ALTER TABLE

- By using alter table command we can modify the structure of the database.
- Using ALTER TABLE statement to:
  - Add a new column
  - Modify an existing column
  - Define a default value for the new column
  - Drop a column
  - Rename column name

fppt.com

Alter table

Add / modify column

```
ALTER TABLE
<table name>
 ADD / MODIFY
(
<column name> data type (size)
);
```

Delete column

```
ALTER TABLE
<table name> DROP
(<column name>);
```

Example

- ALTER TABLE student ADD (Div char(1));
- ALTER TABLE student MODIFY (Name char(12));
- ALTER TABLE student DROP (Div);

fppt.com

Truncate Table :

- This command used to remove all records stored in database.
- It is similar to DROP TABLE command.
- When you place truncate table command your table will be empty.
- Only the difference between drop & truncate is truncate removes all records from table but not remove table definition or structure.

Syntax:

TRUNCATE TABLE <table name>;

Example :

TRUNCATE TABLE student;

fppt.com

Drop Table :

- This command used to remove table definition.
- When you place drop table command it will remove all records first then remove table definition.

Syntax:

DROP TABLE <table name>;

Example :

DROP TABLE student;

fppt.com

RENAME

- This command is used to rename table.

Syntax:

Rename <old table name> TO <new table name> ;

Example :

RENAME student **TO** stud;

fppt.com

DML COMMANDS

Sr.No.	Command & Description
1	SELECT Retrieves certain records from one or more tables.
2	INSERT Creates a record.
3	UPDATE Modifies records.
4	DELETE Deletes records.

fppt.com

SELECT

- To retrieve data stored in our tables, we use the SELECT statement.
- Syntax:

```
SELECT */<fieldname> |  FROM <table_name> /TAB
[ WHERE  <condition> ]
[GROUP BY <fieldname> [HAVING <Condition>]]
[ORDER BY <fieldname>  [ORDER]];
```

- Example

```
SELECT * FROM emp;
SELECT rlno, name, stream FROM stud
WHERE stream ='BSCIT'
ORDER BY rlno DESC;
```

ppt.com

SELECT

- To retrieve data stored in our tables, we use the SELECT statement.
- Syntax:

```
SELECT */<fieldname> |  FROM <table_name> | TAB
[ WHERE  <condition> ]
[GROUP BY <fieldname> [HAVING <Condition>]]
[ORDER BY <fieldname>  [ORDER]];
```

- Example

```
SELECT * FROM emp;
SELECT rlno, name, stream FROM stud
WHERE stream ='BSCIT'
ORDER BY rlno DESC;
```

ppt.com

Insert syntax & example:

- It's a DML command which is used to enter data inside a field of table.

Syntax:

INSERT INTO <table name> [(<field name>,…)] VALUES
(
<value1>,<value2>…..
);

Example:

Insert into stud values(1 ,'john', 'bscit', '01-jan-03');

fppt.com

UPDATE

- Update command used to update the records of a table.
- We can modify existing data with the help of Update.

Syntax:

UPDATE TABLE SET <column_name> = value
[, <column_name> = value….]
[WHERE <condition>];

Example:

UPDATE stud set DIV='B' where rlno=5;

fppt.com

DELETE

- This command is used to remove one or all records from table

Syntax:

DELETE FROM <table_name> [WHERE <condition>];

Example

DELETE FROM stud where rlno=8;

fppt.com

DCL COMMANDS

Sr.No.	Command & Description
1	GRANT Used to assign privileges
2	REVOKE Remove previously assigned privileges.

fppt.com

DCL commands

Grant

- With the help of grant command we can
  - Assign role: connect, resource & DBA.
  - Assign rights to perform Insert data , Update data , Display data or Delete data from a specific table.

syntax & example

- **Syntax:**

GRANT <PRIVILEGE> | ROLE ON [<OBJECT NAME>] TO <USER NAME>;

- **Example:**
  - GRANT CONNECT TO MKB;
  - GRANT INSERT, UPDATE ON STUD TO MKB;

fppt.com

DCL commands

Revoke

- With the help of revoke command we can
  - Remove role: connect, resource & DBA.
  - Remove all rights to perform Insert data , Update data , Display data or Delete data from a specific table.

syntax & example

- **Syntax:**

REVOKE <PRIVILEGE> | ROLE ON [<OBJECT NAME>] FROM <USER NAME>;

- **Example:**
  - REVOKE CONNECT FROM MKB;
  - REVOKE INSERT, UPDATE ON STUD FROM MKB;

fppt.com

TCL COMMANDS

Sr.No.	Command & Description
1	Commit Used to save transaction
2	Rollback Remove the effect of last transaction
3	SAVEPOINT It is essentially allows partial rollbacks within a transaction.

fppt.com

COMMIT COMMAND

- A set of changes made to a table using DML (INSERT, UPDATE, DELETE) commands are temporary. To make the changes permanent, COMMIT command must be needed after DML commands.
- **Syntax: -**

```
COMMIT;
```

- **Example: -**

```
DELETE FROM EMP WHERE NO=500;
COMMIT;
```

fppt.com

ROLLBACK COMMAND

- The ROLLBACK statement allows you to change your mind about a transaction.
- It brings back the original state of the tables to the state as of the last COMMIT statement or the beginning of the current transaction.

- **Syntax: -**

  ROLLBACK [TO SAVEPOINT <Savepoint_Name>];

- **Example: -**

  DELETE FROM EMP WHERE NO=500;

  ROLLBACK;

- **NOTE: -** You cannot ROLLBACK the transaction after the COMMIT. So, After the COMMIT you cannot use ROLLBCAK command.

fppt.com

SAVE POINT

- Additionally, you can use SAVEPOINT to further subdivide the DML statements within a transaction before the final COMMIT of all DML statements within the transaction.
- SAVEPOINT essentially allows partial rollbacks within a transaction.

- **Syntax: -**

  SAVEPOINT <Savepoint_Name>;

  ROLLBACK TO SAVEPOINT <Savepoint_Name>;

- **Example: -**

  DELETE FROM EMP WHERE NO=500;

  SAVEPOINT A1;

  ROLLBACK TO SAVEPOINT A1;

fppt.com

WHAT IS PL/SQL?

- PL/SQL stands for Procedural Language extensions to the Structure
- PL/SQL is a combination of SQL along with the procedural features of programming languages.
- It was developed by Oracle Corporation in the early 90's to enhance the capabilities of SQL.
- Oracle uses a PL/SQL engine to processes the PL/SQL statements.

fppt.com

Advantages of PL/SQL

1) Procedural Capabilities
2) Support to variables
3) Error Handling
4) User Defined Functions
5) Portability
6) Sharing of Code
7) Efficient Execution

fppt.com

Advantages of PL/SQL

1) Procedural Capabilities

- PL/SQL provides procedural capabilities such as **condition checking, branching and looping.**
- This enables programmer to **control execution** of a program based on some conditions and user inputs.

2) Support to variables

- PL/SQL supports **declaration and use of variables**.
- These variables can be used to **store intermediate results** of a query or some expression.

3) Error Handling

- When an error occurs, **user friendly message can be displayed**.
- **Execution of program can be controlled** instead of abruptly terminating the program.

fppt.com

Advantages of PL/SQL

4) User Defined Functions

- SQL also supports **user defined functions and procedures**.

5) Portability

- Programs written in PL/SQL are portable.
- It means, programs can be **transferred and executed from any other computer hardware and operating system, where Oracle is operational.**

6) Sharing of Code:

- Allows user to **store compiled code** in database.
- PL/SQL code can be **accessed and shared by different applications.**
- PL/SQL code can be **executed by other programming language** like JAVA.

fppt.com

Advantages of PL/SQL

7) Efficient Execution

- PL/SQL sends an entire block of SQL statements to the Oracle engine, where these statements are executed in one go.
- Reduces **network traffic and improves efficiency of execution**.
- In **SQL, all statements are transferred one by one.**

fppt.com

SQL VS PL/SQL

SQL	PL/SQL
SQL stands for structured query language	PL/SQL stands procedural language/ structure query language.
Can not use variables, constants	Use variables & constants
Allow you to use built in functions only	Allow you to use built in functions as well as create your own user define function
SQL commands can not be sharable	You can share PL/SQL code
In SQL we can not use conditional statement & looping statement	In PL/SQL we can use conditional statement & looping statement
We can use commands to solve queries	We can write a program/group of statements
SQL is syntax/statement oriented language	PL/SQL is structure oriented language

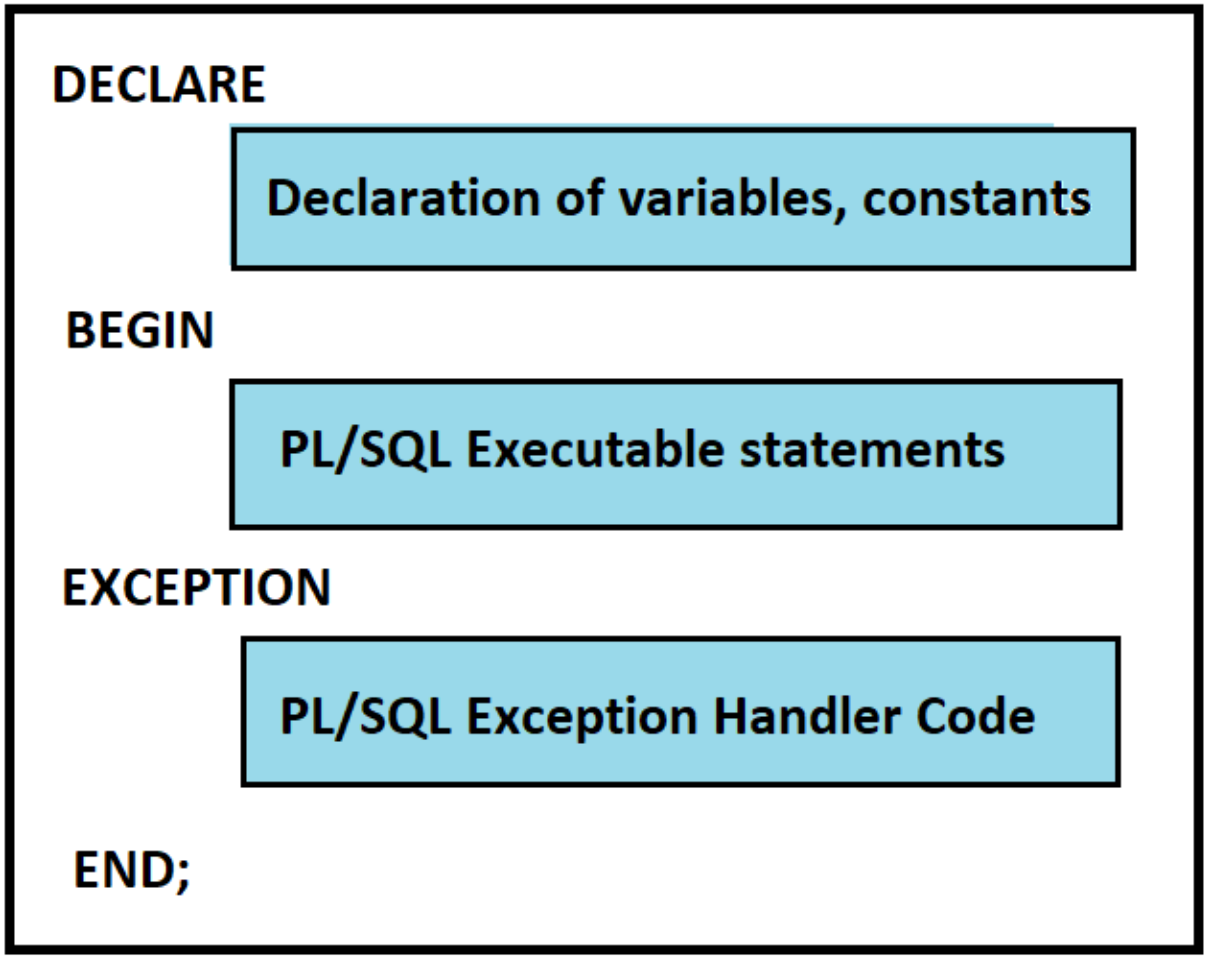
ippt.com

SQL VS PL/SQL

SQL	PL/SQL
In SQL you can not create or use package	Create a package
There is no facility to handle exceptions	Allows you to write exception handling code
There is no facility of trigger.	You can create trigger.
SQL support DML & DDL compiler	PL/SQL is compiled with PL/SQL compiler

fppt.com

Generic PL/SQL Block



fppt.com

Generic PL/SQL Block

1) Declarations (Optional)

- This section starts with the keyword **'DECLARE'.**
- It defines and initializes **variables and cursors used in the block.**

2) Executable Part (Mandatory)

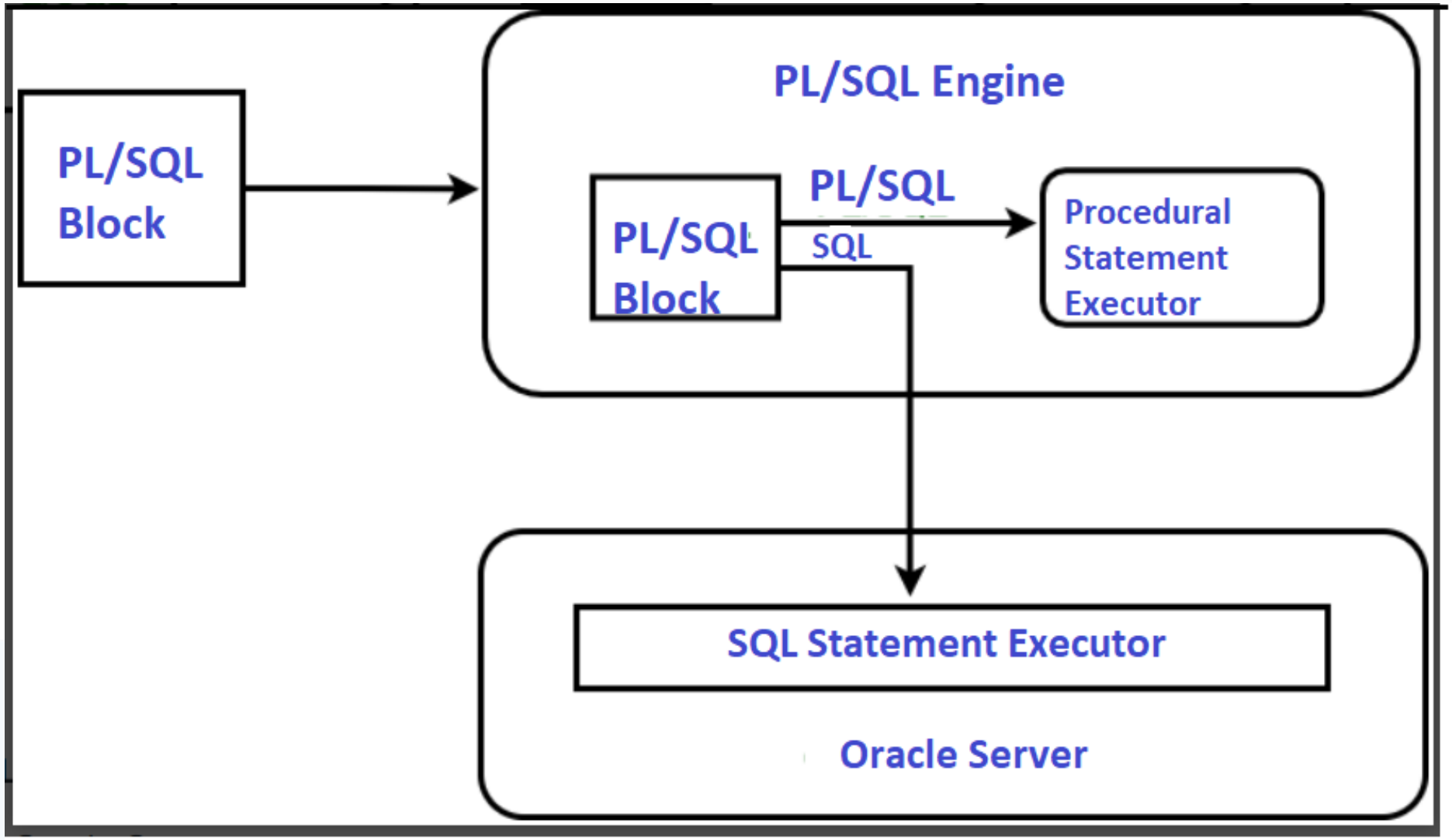
- This section starts with the keyword **'BEGIN'.**
- It contains various SQL and PL/SQL statements providing functionalities like **data retrieval, manipulation, looping and branching.**

3) Exception Handling (Optional)

- This section starts with the keyword **'EXCEPTION'.**
- It **handles errors** that arise during the **execution of data manipulation statements in 'executable commands' section.**
- This section specifies that **end** of PL / SQL block.

fppt.com

PL/SQL Environment



PL/SQL Environment

- The PL/SQL engine accepts as input any valid PL/SQL unit.
- The engine runs procedural statements, but sends SQL statements to the SQL engine in the database.
- Typically, the database processes PL/SQL units.
- When an application development tool processes PL/SQL units, it passes them to its local PL/SQL engine.
- If a PL/SQL unit contains no SQL statements, the local engine processes the entire PL/SQL unit.
- This is useful if the application development tool can benefit from conditional and iterative control.

fppt.com

The PL/SQL Literals

- A literal is an explicit numeric, character, string, or Boolean value not represented by an identifier.
- For example, TRUE, 786, NULL, ‘marwadi university' are all literals of type Boolean, number, or string.
- PL/SQL, literals are case-sensitive. PL/SQL supports the following kinds of literals –
  - Numeric Literals
  - Character Literals
  - String Literals
  - BOOLEAN Literals
  - Date and Time Literals

fppt.com

The PL/SQL Literals

- **Numeric Literals:**

25	6.34	7g2	.1	+17	-5
050	78	-14	0	+32767	
6.6667	0.0	-12.0	3.14159	+7800.00	

- **Character Literals**

'A'	'%'	'*'	'z'	'('

fppt.com

The PL/SQL Literals

- **String Literals**
- 'Hello, world!'
- ‘Marwadi Univerisity'
- '19-NOV-12'

- **BOOLEAN Literals**
- TRUE, FALSE, and NULL.

- **Date and Time Literals**
- DATE ‘25-DEC-78';
- TIMESTAMP '2012-10-29 12:01:01';

fppt.com

Data Types

- PL/SQL is **super set** of the SQL.
- So, it supports all the data types provided by SQL.
- In PL/SQL Oracle provides **subtypes of the data types.**
- **For example,** the data type NUMBER has a subtype called INTEGER.

fppt.com

Data Types

Category	Data Type	Sub types/values
Numerical	NUMBER	BINARY_INTEGER, DEC, DECIMAL, DOUBLE PRECISION, FLOAT, INTEGER, INT, NATURAL, POSITIVE, REAL, SMALLINT
Character	CHAR,LONG, VARCHAR2	CHARACTER, VARCHAR, STRING, NCHAR, NVARCHAR2
Date	DATE	-
Binary	RAW, LONG RAW	-
Boolean	BOOLEAN	Can have value like TRUE, FALSE and NULL
RowID	**ROWID**	**Stores values of address of each record**

fppt.com

PL/SQL CONSTANT

- A constant holds a value that once declared, does not change in the program.
- A constant declaration specifies its name, data type, and value, and allocates storage for it.
- The declaration can also impose the **not null constraint.**
- A constant is declared using the **constant** keyword.
- Syntax:

  <constant name> CONSTATNT <datatype>:=value;

- E.G.

  PI **CONSTATNT** number:=3.14;

fppt.com

Variables

- Variables are used to store values that may be change during the execution of program.
- In PL/SQL, variables contain values resulting from **queries or expression**.
- Variables are declared in **Declaration section** of the PL/SQL block.
- It can assign valid data type and can also be initialized if necessary.

fppt.com

Variables

Declare a Variable

Syntax:

variableName datatype [NOT NULL] := initialValue;

- **datatype** can be any valid data type supported by PL/SQL.
- **':= '** used as **assignment operator.**
- If variable need to be initialized then **initialValue** can be assigned at declaration time.
- If **NOT NULL** is included in declaration, variable cannot have NULL value during program execution and such **variable must be initialized.**

fppt.com

Variables

Example:

```
no   NUMBER(3);
value   DECIMAL;
city   CHAR(10);
name   VARCHAR(10);
counter   NUMBER(2)   NOTNULL  :=  0
```


fppt.com

Anchored Data Type Variable

- A variable can be declared as **anchored data type**.
- It means, **datatype** for variable is **determined based on the data type of the other object.**
- This object can be other **variable or a column of the table.**
- This provides ability to match the **data types of the variables** with the **data types of the columns** defined in the database.
- If data type of column is changed, then the data type of variable will also **changed automatically**.

Advantage: It **reduces maintenance cost** and allow a program to adapt changes made in tables.

fppt.com

Anchored Data Type Variables

Syntax:

variableName object%TYPE [NOTNULL] := initialValue ;

- **object** can be any variable declared previously or column of a database.
- To refer of a column of particular table, column name must be combined with table name.

Example

no **Account.Acc_No%TYPE := 101;**

bal **Account.Balance%TYPE;**

name **Customer.name%TYPE;**

fppt.com

Assign Value to Variable

1) **By using Assignment Operator**

   Syntax:

   variableName **:=** **value** ;

   - A value can be a **constant value or result of some expression or return value of some function.**

2) **By Reading from the Keyword:**

   Syntax:

   variableName **:=** **&variableName** ;

   - This is similar to **scanf() function.**
   - Whenever **'&'** is encountered, a **value read from the keyboard** and assign it to variable.

   no := &no;

fppt.com

Assign Value to Variable

3) **Selecting or Fetching table data values into Variables:**

Syntax:

SELECT col1, col2, …, colN **INTO** var1, var2, …, varN **FROM** tableName **WHERE** condition ;

- This statement retrieves values for **specified column and stores** them in given variable.
- **Data type and size of variables** must be **compatible with the relative columns**.
- A condition in **WHERE** clause must be such that it selects **only single record**.
- This statement cannot work if **multiple records are selected**.

fppt.com

Assign Value to Variable

3) **Selecting or Fetching table data values into Variables:**

Example: Write PL/SQL block stores account number and balance for account '**A01**' into variable '**no**' and '**bal**'.

Input:

```
DECLARE
        no      Account.Acc_No%TYPE;
        bal     Account.Balance%TYPE;
BEGIN
        SELECT  Acc_No, Balance  INTO  no, bal  FROM
Account                  WHERE  Acc_No = 'A01' ;
END;
/
```


fppt.com

Display Messages

- **Syntax:**
- **dbms_output.put_line (message);**
- A **dbms_output** is a **package**, which provides functions to accumulate information in a buffer.
- A **put_line** is a **function**, which display messages on the screen.
- A **message** is a **character string to be displayed**.
- To display **data of other data type**, they must be **concatenated with some character string**.
- The environment parameter, **SERVEROUTPUT** must be **ON** to display messages on screen.
- **SET SERVEROUTPUT ON;**

fppt.com

Display Messages

Example:

```
dbms_output.put_line ( 'Hi Hello World...' );
```

Hi Hello World...

```
dbms_output.put_line ( 'Sum =' || 25 );
```

Concatenation Operator

Sum = 25

```
dbms_output.put_line ( 'Square of ' || 3 || ' is ' || 9 );
```

Square of 3 is 9

fppt.com

Comments

- Comments are statement that will **not get executed** even though they are present in the program code.
- Comments are used to **increase readability** of a program.
- There are two types of comments:

1) -- (Double hyphen or double dash) (Single Line Comment):

- Treats single line as a comment.

```
--  This single line is a comment
```

2) /* …. */ (Multiple Line Comment):

- Treats multiple lines as comment.

```
/*  This statement is spread over two line and both lines
are treated as comments  */
```


fppt.com

Control Structures

- There are 3 types of Control Structures in PL/SQL:

1) **Conditional Control**

2) **Iterative Control**

3) **Sequential Control**

fppt.com

Control Structures - Conditional Control

- To control the **execution of block of code** based on some condition, PL/SQL provides the **IF statement.**
- Three form of conditional controls
  - Simple if
  - Else if
  - If ….then….elseif ….else

fppt.com

Conditional Control

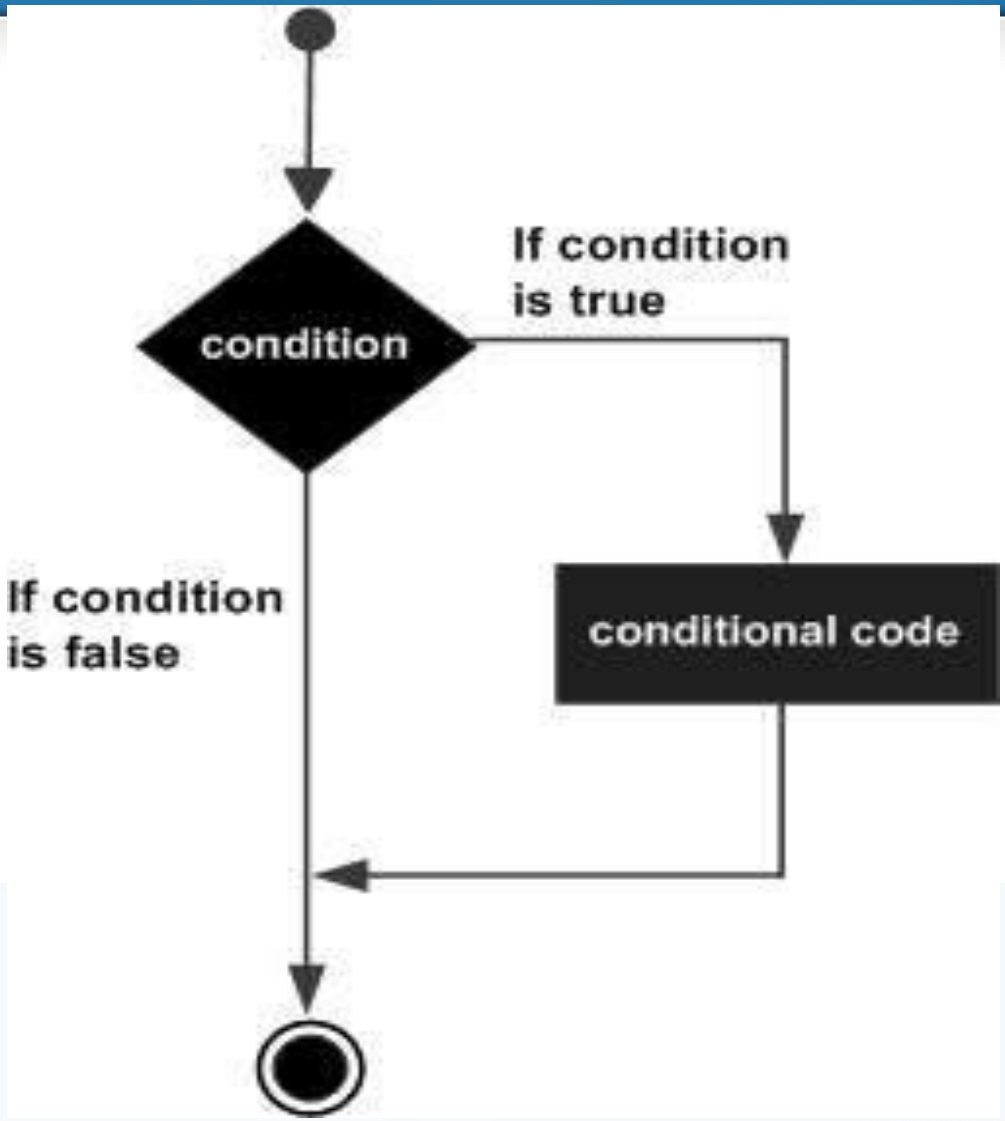
- Simple if

Syntax

If <condition> Then

<statements>

End if;



fppt.com

Conditional Control

- If …else

Syntax

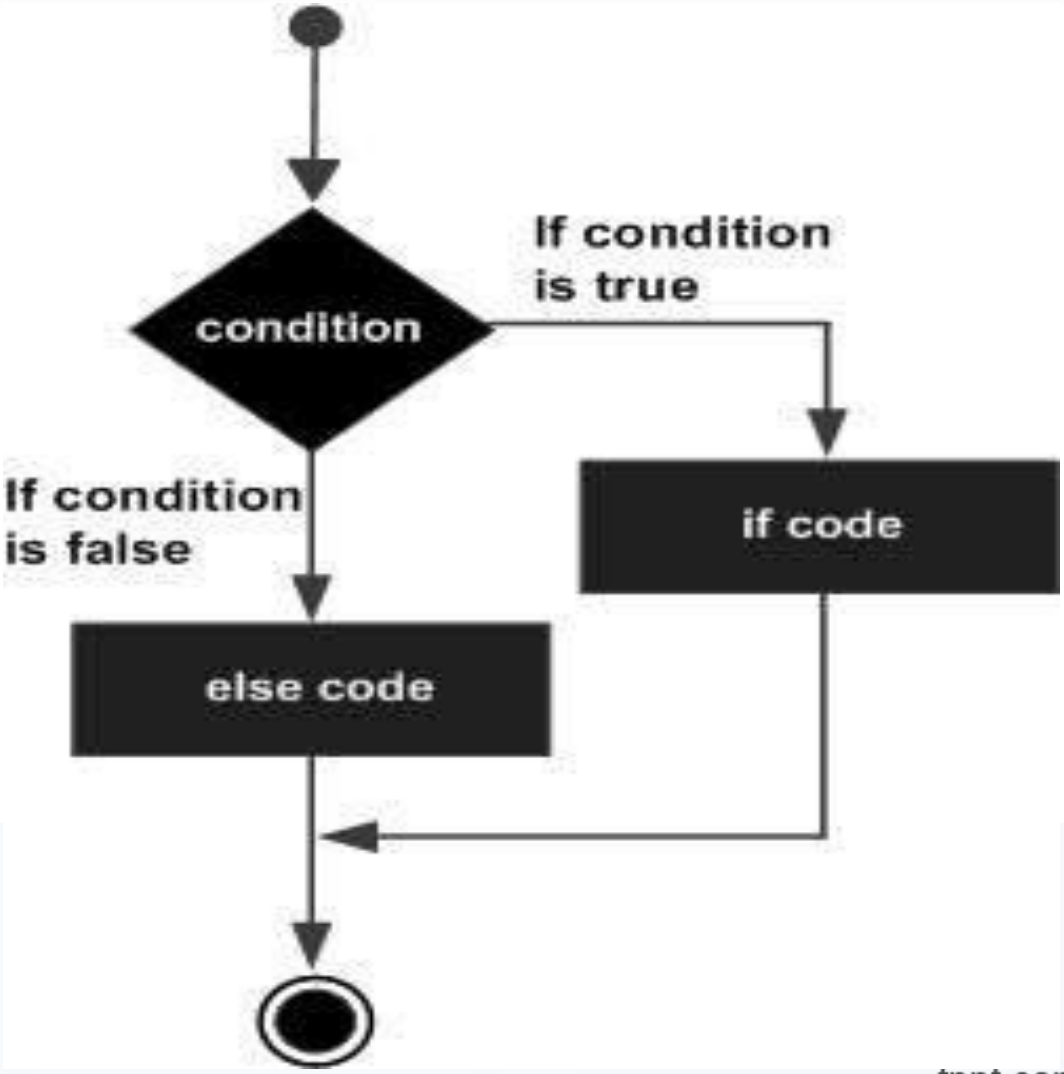
If <condition> Then

<statement 1>

Else

<Stement2>

End if;



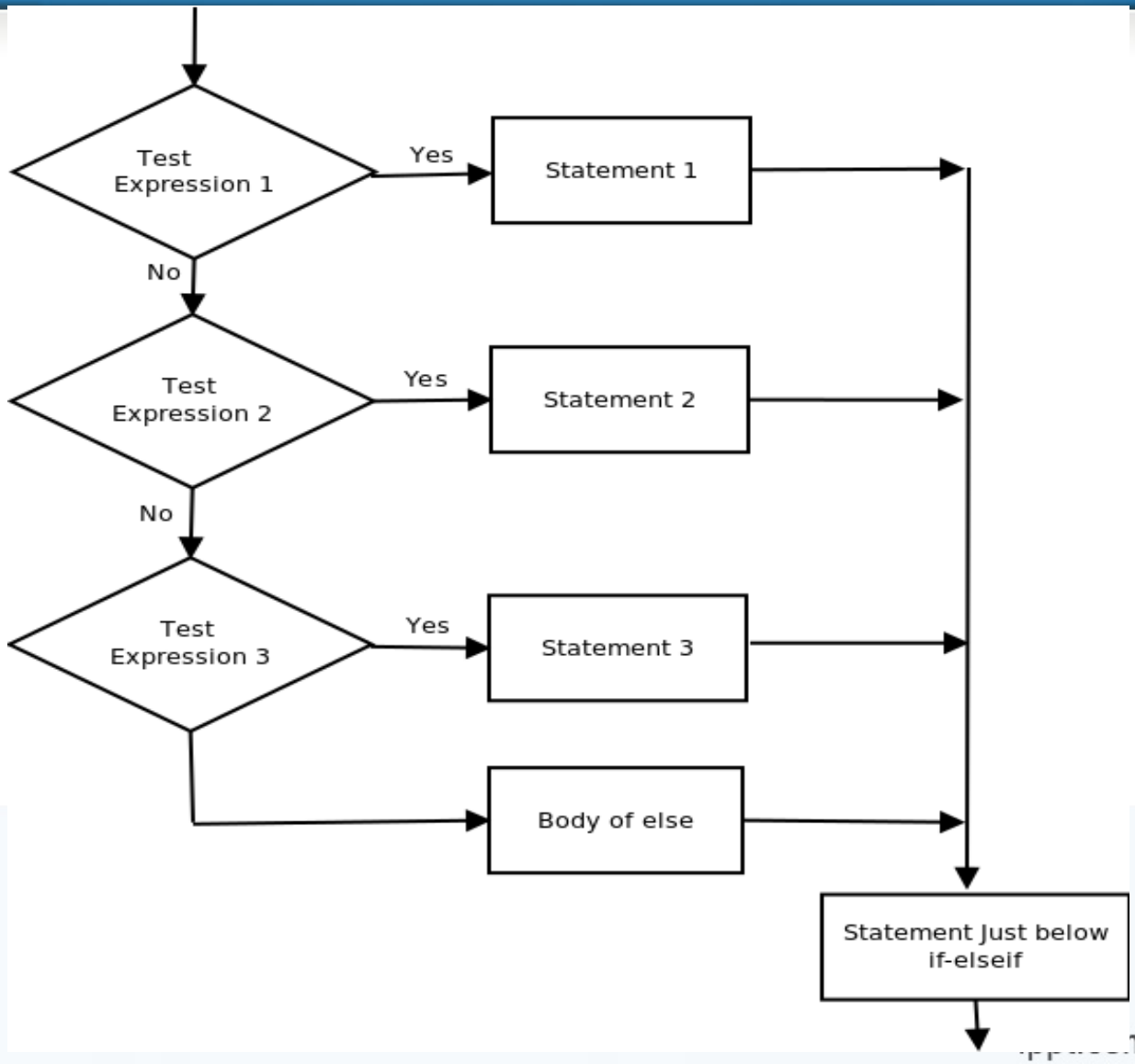
tppt.com

Conditional Control

- If …then…elsif

```
If <Condition> then
        <Statement1>
Elsif <condition2>
then
        <statment2>
Elsif <condition3>
Then
        <Stement3>
Else
                …

End if;
```



Control Structures- Iterative Control

- Iterative control allows a group of statements to execute **repeatedly in a program.** It is called **Looping**.
- PL/SQL provides three constructs to implement loops, as listed below:
  - **1. LOOP**
  - **2. WHILE**
  - **3. FOR**
- In PL/SQL, any loop starts with a **LOOP** keyword and it terminates with an **END LOOP** keyword.
- Each loop requires a **conditional statement** to control the **number of times a loop** is executed.

fppt.com

Iterative control- Loop

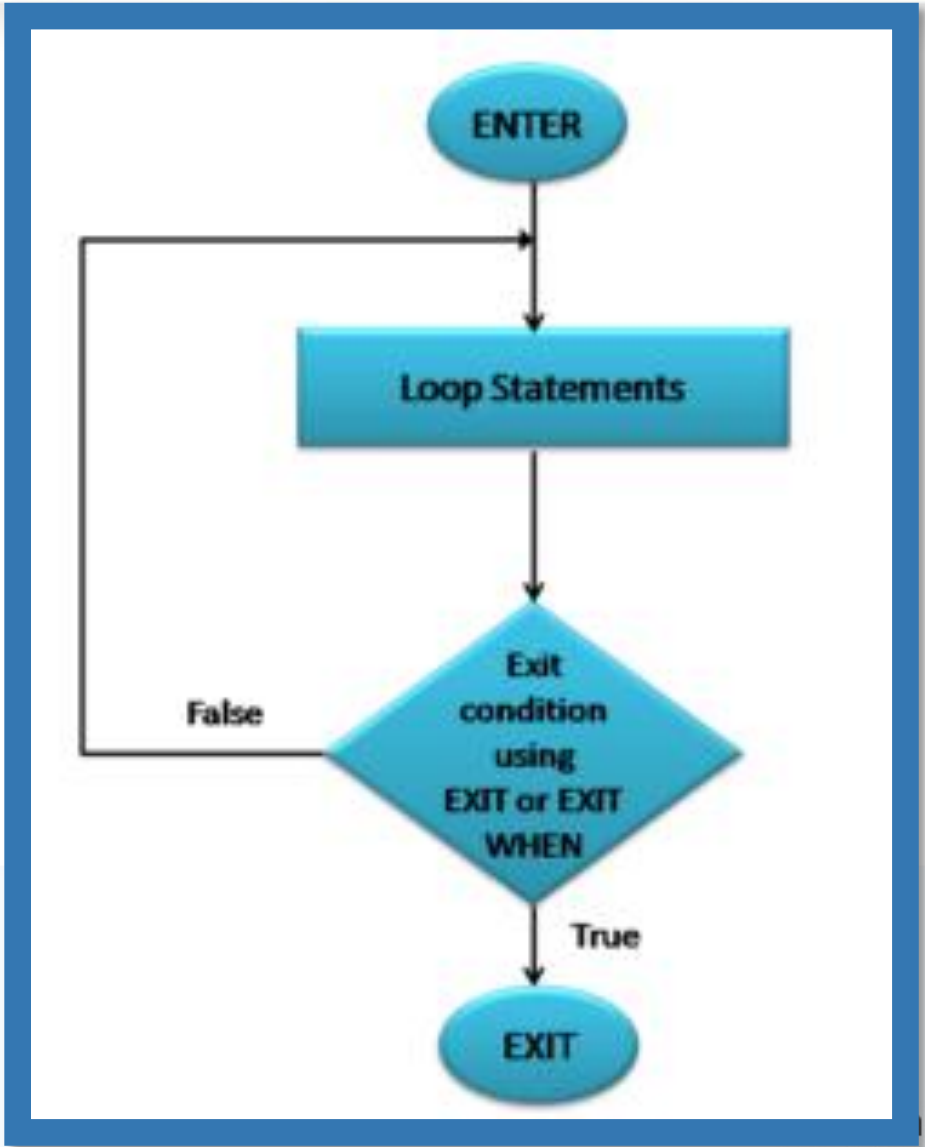
Syntax:

Loop

<Statements>;

[Exit When <Condition>];

End Loop;

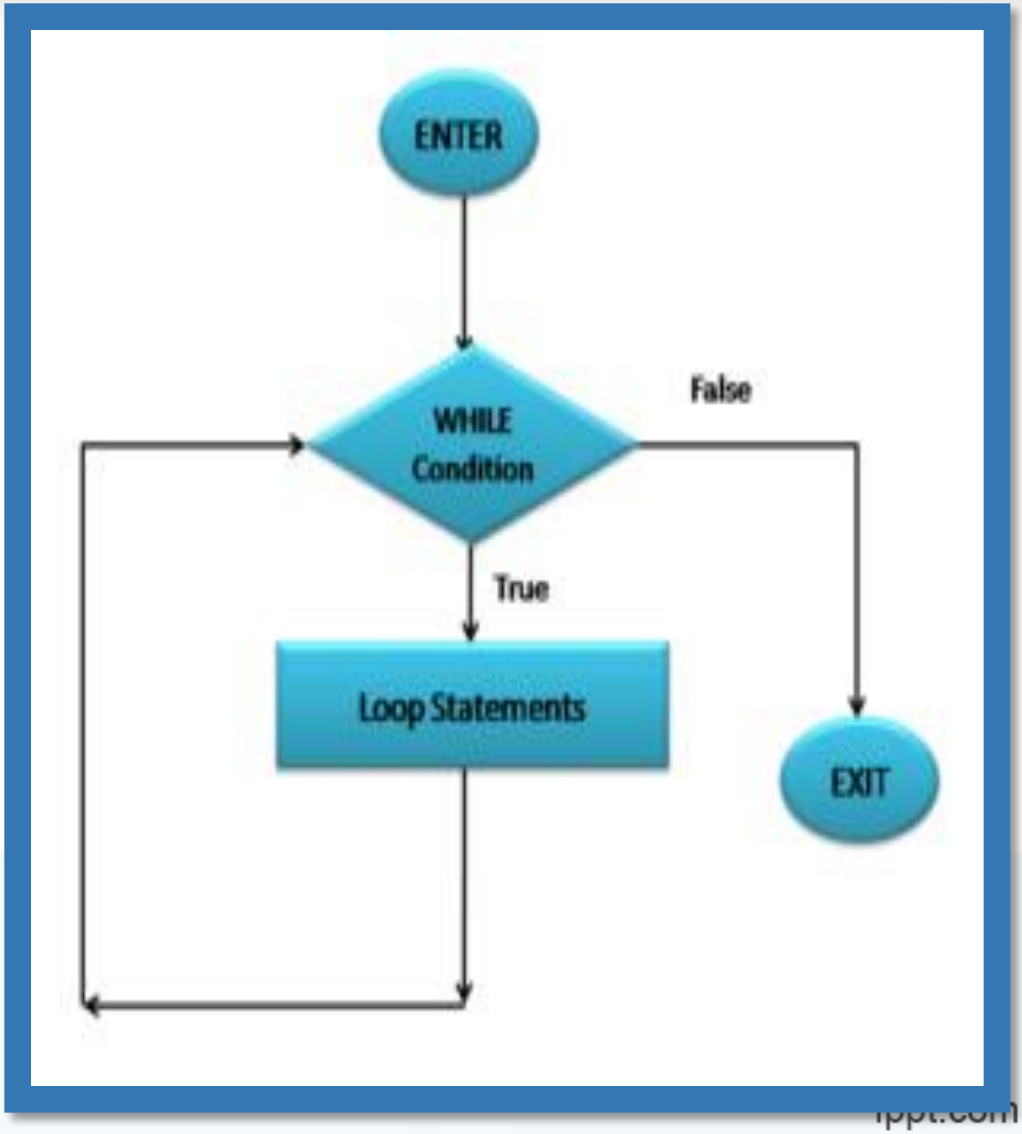


Iterative control- While

Syntax:

While <condition>
Loop

<Statements>;
End Loop;



Iterative control- for

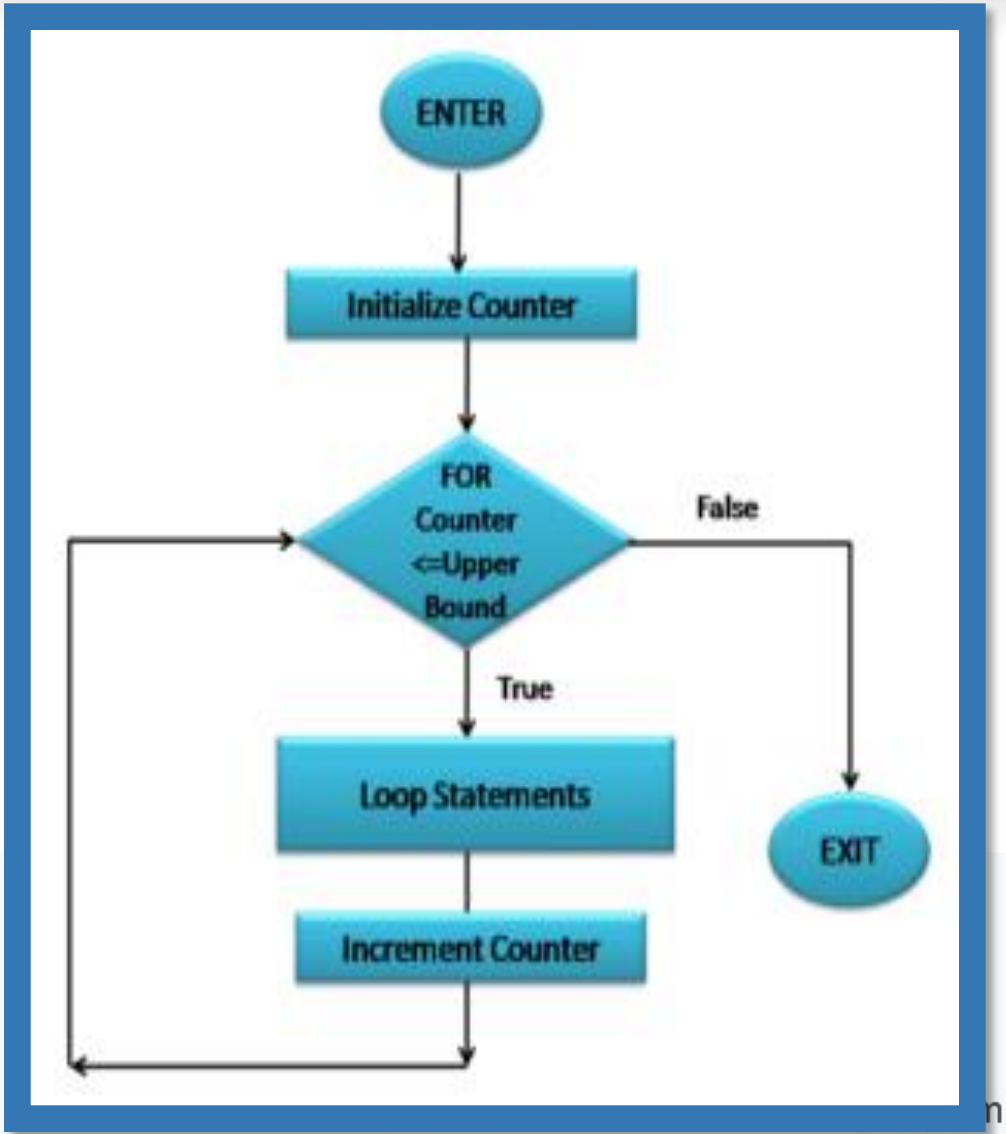
Syntax:

For Variable In [Reverse]
Start..End

Loop

<Statement>

End Loop;



Control Structures- sequential control

- Normally, execution proceeds sequentially within the block of code.
- Sequence can be changed conditionally as well as **unconditionally**.
- To alter the sequence **unconditionally**, the **GOTO statement** can be used.

Syntax:

```
GOTO    jumpHere ;
          :
          :
<< jumpHere >>
```

- The **GOTO statement** makes flow of execution to jump at **<< jumpHere >>**.

fppt.com

Control Structures- sequential control

```
BEGIN
GOTO second_message; <<first_message>>
        DBMS_OU TPUT.PUT_LINE( 'Hello' );
GOTO the_end;
<<second_message>>
        DBMS_OUTPUT.PUT_LINE( 'PL/SQL GOTO
Demo' );
GOTO first_message;
<<the_end>>
        DBMS_OUTPUT.PUT_LINE( 'and good bye...'
);
END;
```

